Projekt

PSIR

Autor:

Kajetan Wilkowski 331542

Jakub Surma 327768

Albert Kołsut 331493

12 stycznia 2026

WYDZIAŁ ELEKTRONIKI
I TECHNIK INFORMACYJNYCH

Spis treści

1 Definicja Zadania Projektowego **2**
1.1 Wymagania dotyczące protokołu ALP (Application Layer Protocol) . . . 2
1.2 Wymagania funkcjonalne Serwera . . . 2
1.3 Wymagania funkcjonalne Węzłów (Nodes) . . . 2

2 Wykorzystane narzędzia **3**

3 Implementacja Protokołu ALP **3**
3.1 Specyfikacja formatu ramki i mapowanie znaków . . . 3
3.1.1 Separatory binarne . . . 3
3.1.2 Mapowanie symboli derywacji (interpretacja żółwia) . . . 4
3.2 Katalog i semantyka wiadomości . . . 4
3.2.1 Komunikaty sterujące (Kontrola stanu) . . . 4
3.2.2 Komunikaty operacyjne (Dane) . . . 4
3.2.3 Komunikaty Handover (Przekazanie sterowania) . . . 4
3.2.4 Komunikaty Zakończenia (Terminacja sesji) . . . 4
3.3 Spełnienie wymagań projektowych . . . 5

4 Implementacja Serwera **5**
4.1 Procedura działania systemu . . . 5
4.2 Opis struktur i funkcji . . . 6
4.2.1 Struktury danych . . . 6
4.2.2 Kluczowe funkcje logiczne . . . 6
4.3 Spełnienie wymagań projektowych . . . 7

5 Implementacja Węzła IoT **7**
5.1 Procedura działania węzła . . . 7
5.2 Opis struktur i funkcji . . . 8
5.2.1 Kontekst operacyjny i struktury danych . . . 8
5.2.2 Szczegółowy opis kluczowych funkcji logicznych . . . 8
5.3 Spełnienie wymagań projektowych . . . 8

6 Działanie systemu **9**
6.1 Sposoby wywołania . . . 9
6.1.1 Wywołanie z plikiem Lsystem.txt . . . 10
6.1.2 Wywołanie z użyciem konsoli . . . 10
6.2 Wyniki . . . 11
6.3 Testy odporności na błędy . . . 13
6.3.1 Zabezpieczenie infile.txt . . . 13
6.3.2 Zabezpieczenie przed podawaniem nieprawidłowych parametrów L-sytemu . . . 13
6.3.3 Rozłączenie jednego z węzłów . . . 13
6.3.4 Wykorzystanie wszystkich symboli derywacji L-systemu . . . 14
6.4 Wyjście żółwia poza obszar całkowity . . . 15

7 Podsumowanie **15**

1 Definicja Zadania Projektowego

Celem projektu było stworzenie rozproszonej aplikacji służącej do renderowania słów generowanych przez systemy Lindenmayera (L-systemy) przy użyciu grafiki żółwia (turtle graphics). Obraz wynikowy miał być generowany w formacie ASCII art, gdzie jeden znak odpowiada jednemu pikselowi (lub wielokrotności), przy wykorzystaniu czcionki monospaced. Dodatkowo każda część implementacji miała oddzielne wymagania, które musieliśmy spełnić.

1.1 Wymagania dotyczące protokołu ALP (Application Layer Protocol)

Protokół ALP musi być autorskim rozwiązaniem służącym do komunikacji między serwerem a węzłami (Nodes).

- **Charakter binarny** - ALP musi być protokołem binarnym w celu oszczędności pamięci oraz pasma transmisyjnego.
- **Definicja na poziomie bitowym** - Należało zidentyfikować pola bitowe wewnątrz słów oraz określić kodowanie dla każdego pola.
- **Niezależność** - Protokół musi być niezależny od języka programowania oraz platformy sprzętowej.
- **Warstwa transportowa** - ALP musi działać w oparciu o bezpołączeniowy protokół UDP.
- **Mechanizm niezawodności** - Ze względu na zawodność protokołu UDP, w ALP należy zaimplementować własny schemat zapewnienia niezawodności.
- **Dokumentacja** - Formatu wiadomości ALP musi zostać udokumentowany w sposób czytelny.

1.2 Wymagania funkcjonalne Serwera

Serwer pełni rolę jednostki centralnej koordynującej proces renderowania.

- **Obsługa wejścia** - Pobieranie definicji L-systemu oraz długości derywacji z linii komend lub z pliku.
- **Zarządzanie węzłami** - Przyjmowanie rejestracji od uczestniczących w systemie węzłów IoT.
- **Partycjonowanie przestrzeni** - Przydzielanie każdemu zarejestrowanemu węzłowi danego regionu płaszczyzny renderowania.
- **Dystrybucja zadań** - Przekazywanie niezbędnych danych do węzłów.
- **Koordynacja renderowania** - Inicjacja procesu rysowania oraz obsługa przekazań sterowania (handovers) między węzłami.
- **Agregacja wyników** - Zbieranie fragmentów wygenerowanego obrazu od węzłów i złożenie ich w jedną całość.
- **Prezentacja** - Wyświetlanie kompletnego obrazu ASCII art.
- **Diagnostyka** - Ciągłe aktualizowanie metryk oraz generowanie komunikatów diagnostycznych.
- **Ograniczenia implementacyjne** - Wykorzystanie wyłącznie systemowego API Linux oraz języka C.

1.3 Wymagania funkcjonalne Węzłów (Nodes)

Węzły wykonują faktyczną pracę obliczeniową i interpretację graficzną.

- **Rejestracja** - Samodzielne zgłoszenie gotowości do pracy do serwera.
- **Konfiguracja** - Odbiór przydzielonego podregionu oraz parametrów interpretacji żółwia.
- **Pliki środowiskowe** - Odczytywanie współrzędnych początku układu derywacji żółwia z lokalnych plików konfiguracyjnych.

- **Renderowanie** - Wykonywanie obliczeń grafiki żółwia dla przydzielonego zadania w ramach swojego subregionu.
- **Obsługa granic** - Wykrywanie sytuacji wyjścia żółwia poza przydzielony obszar (handover) i raportowanie tego faktu do serwera.
- **Zakończenie pracy** - Wykrywanie końca procesu renderowania i przesyłanie wygenerowanego fragmentu obrazu (ASCII art) do serwera.
- **Ograniczenia implementacyjne** - Implementacja w środowisku emulatora Arduino przy użyciu standardowego API Arduino oraz rozszerzeń PSIR.

2 Wykorzystane narzędzia

W celu opracowania naszego rozwiązania posługiwaliśmy się głównie narzędziami znanymi nam z laboratoriów:

- **Języki programowania:**
  - Język C - Wykorzystany do napisania kodu Serwera.
  - Język C++ (Standard Arduino) - Wykorzystany do implementacji logiki węzłów w środowisku Arduino.
- **Platformy i aplikacje:**
  - Oracle VirtualBox - Wykorzystywany do obsługi maszyn wirtualnych psir23Z, dostarczonych przez prowadzących, działających na systemie operacyjnym Linux.
  - Arduino IDE - Platforma użyta do napisania kodów węzłów oraz eksportowania skompilowanych plików do uruchomienia na maszynach wirtualnych
  - Putty/MobaXterm - Aplikacje używane do tworzenia połączeń SSH z maszynami wirtualnymi w celu łatwiejszej obsługi i importu plików

3 Implementacja Protokołu ALP

Protokół ALP jest jednym z 3 kluczowych elementów tworzonego przez nas systemu. Jest on zaprojektowany w architekturze Master-Slave i służy do koordynacji rozproszonego procesu renderowania L-systemów. Komunikacja odbywa się w trybie asynchronicznym, ciągłę zapewniając synchronizację stanu żółwia między serwerem a węzłami.

3.1 Specyfikacja formatu ramki i mapowanie znaków

Każda wiadomość protokołu ALP składa się z nagłówka (typu) oraz opcjonalnej części danych, oddzielonych separatorami o stałym znaczeniu binarnym.

3.1.1 Separatory binarne

- `|` (ASCII 0x7C) - Oddziela nazwę typu wiadomości od sekcji pól danych.
- `;` (ASCII 0x3B) - Oddziela poszczególne pola danych od siebie.
- `=` (ASCII 0x3D) - Oddziela klucz (nazwę pola) od jego wartości.
- `:` (ASCII 0x3A) - Używany jako separator wewnątrz wartości złożonych (np. współrzędne `x:y:a`).

3.1.2 Mapowanie symboli derywacji (interpretacja żółwia)

Każdy symbol L-systemu przesyłany w polu `symbol=` jest mapowany na 8-bitowy kod ASCII, co zapewnia jednoznaczną interpretację przez węzeł:

Symbol	Znaczenie	Kod ASCII	Kod HEX
F	Ruch do przodu z rysowaniem	01000110	0x46
f	Ruch do przodu (skok bez rysowania)	01100110	0x66
+	Obrót w prawo (o zadaną deltę)	00101011	0x2B
-	Obrót w lewo (o zadaną deltę)	00101101	0x2D
|	Obrót o 180 stopni	01111100	0x7C
[	Zapisanie stanu na stos (Push)	01011011	0x5B
]	Przywrócenie stanu ze stosu (Pop)	01011101	0x5D

Tabela 1: Mapowanie bitowe symboli derywacji w protokole ALP

3.2 Katalog i semantyka wiadomości

3.2.1 Komunikaty sterujące (Kontrola stanu)

- `REGISTER_NODE|id=<nazwa>` - Wprowadza obecność węzła w systemie.
- `ASSIGN_REGION|region=<idx>:<xmin>:<ymin>:<xmax>:<ymax>` - Definiuje prostokątny obszar roboczy węzła. Pole `region` pozwala węzłowi samodzielnie decydować o tym kiedy wykonywać handover.
- `INIT_RENDERING|start=<x>:<y>:<a>;step=<d>;angle=<delta>;canvas=<W>x<H>` - Przesyła ogólną konfigurację. Pole `canvas` definiuje granice całego obrazu ASCII, zapobiegając błędom.

3.2.2 Komunikaty operacyjne (Dane)

- `RENDER_STEP|index=<n>;symbol=<s>` - Rozkaz wykonania kroku. Pole `index` pozwala węzłowi zignorować zduplikowane pakiety, które zostały już przetworzone.
- `STEP_DONE|id=<nazwa>;index=<n>` - Potwierdzenie sukcesu. Serwer nie przesunie kursora derywacji bez otrzymania tego komunikatu.
- `PLOT|x=<x>;y=<y>` - Przesyła współrzędne punktu do narysowania. Serwer używa tych danych do nanoszenia znaku # na wynikową mapę bitową.

3.2.3 Komunikaty Handover (Przekazanie sterowania)

- `HANDOVER_REQUEST|index=<n>;next=<nx>:<ny>;state=<x>:<y>:<a>` - Węzeł zgłasza wyjście żółwia poza swój region. Pole `next` wskazuje serwerowi, który węzeł powinien być następny, a pole `state` podaje aktualną pozycję i kąt.
- `HANDOVER_START|state=<x>:<y>:<a>;index=<n>` - Serwer aktywuje nowy węzeł, przekazując mu stan żółwia otrzymany od poprzednika.

3.2.4 Komunikaty Zakończenia (Terminacja sesji)

- `RENDER_DONE|id=server` - Wiadomość wysyłana przez serwer w trybie rozgłoszeniowym do wszystkich węzłów po zakończeniu całej procedury derywacji lub w przypadku błędu krytycznego (np. wyjścia żółwia poza cały obszar).
- `RENDER_DONE|id=<nazwa_węzła>` - Odpowiedź wysyłana przez węzeł do serwera. Dopiero po odebraniu tego pakietu od wszystkich 4 węzłów, serwer zamyka plik `output.txt` i wyświetla końcowe metryki.

3.3 Spełnienie wymagań projektowych

W opracowanym przez nas protokole ALP zaimplementowaliśmy mechanizmy spełniające wymagania techniczne zawarte w specyfikacji projektu:

- **Wykorzystanie protokołu UDP:** Protokół ALP operuje bezpośrednio na warstwie transportowej UDP. Serwer nasłuchuje na porcie 5000, natomiast węzły na porcie 5001. Pozwala to na małozasobową komunikację, niezbędną w systemach IoT o ograniczonych zasobach.
- **Struktura wiadomości i wydajność:** W tym celu zastosowaliśmy format ramek typu: `TYP_WIADOMOSCI|klucz=wartosc;`.
  - **Niezależność od platformy** - Zastosowanie kodowania opartego na standardzie ASCII dla wartości numerycznych i symboli pozwala nam ominąć problemy z różną kolejnością bajtów między procesorem x86 serwera a mikrokontrolerami węzłów.
  - **Oszczędność pamięci** - Zaprojektowaliśmy pola aby byłykrótkie i zwięzłe (np. `x=10;y=20`), co minimalizuje rozmiar pakietu UDP. Każdy symbol L-systemu zajmuje dokładnie 1 bajt.
- **Mechanizm niezawodności (Reliability):** Wymóg niezawodności na łączu UDP został spełniony poprzez implementację modelu **Stop-and-Wait** na poziomie derywacji:
  - Każda wiadomość `RENDER_STEP` wymaga potwierdzenia wiadomością `STEP_DONE`.
  - Jeśli potwierdzenie nie dotrze w ciągu `RETX_TIMEOUT_MS` (300 ms), pakiet jest wysyłany ponownie aż do 30 prób.

4 Implementacja Serwera

Zgodnie z wymaganiami serwer został zaimplementowany w języku C jako aplikacja konsolowa działająca w środowisku Linux. Pełni on rolę zarządcy, który steruje cyklem derywacji, rozdziela zadania dla poszczególnych węzłów oraz zapewni spójność stanu żółwia w systemie rozproszonym.

4.1 Procedura działania systemu

Działanie serwera podzielone jest na pięć głównych faz logicznych:

1. **Inicjalizacja i konfiguracja:**
   - Wczytanie definicji L-systemu (aksjomat, reguły, iteracje) z pliku, bądź konsoli.
   - Wykonanie pełnej derywacji słowa za pomocą funkcji `derive_lsystem`.
   - Odczyt parametrów startowych (pozycja x, y, kąt) z pliku `infile.txt`.
   - Utworzenie gniazda UDP i związanie go z portem 5000.
2. **Rejestracja węzłów:** Serwer nie wykonuje dalszej akcji do momentu odebrania komunikatów `REGISTER_NODE` od wszystkich 4 węzłów. Każdy węzeł jest zapisywany w tablicy struktur wraz z jego adresem sieciowym.
3. **Podział regionów oraz rozpoczęcie renderowania:**
   - Obliczenie podziału płótna na regiony (`compute_regions`).
   - Rozesłanie do każdego węzła informacji o jego granicach (`ASSIGN_REGION`) oraz parametrach globalnych (`INIT_RENDERING`).
   - Wyznaczenie węzła startowego i aktywacja go za pomocą komunikatu `HANDOVER_START`.
4. **Proces renderowania:** Serwer wchodzi w główną pętlę opartą na funkcji `select()`, obsługując następujące zdarzenia:
   - **Retransmisja** - Jeśli minął czas `RETX_TIMEOUT_MS` bez otrzymania potwierdzenia kroku, serwer ponawia wysłanie.

- **Aktualizacja stanu** - Po komunikacie `STEP_DONE` serwer przesuwa indeks derywacji i aktualizuje lokalny stan żółwia.
- **Obsługa Handoveru** - Przy `HANDOVER_REQUEST` serwer przelicza, który węzeł powinien przejąć sterowanie i przekazuje mu aktualny stan.

5. **Finalizacja:** W tej fazie serwer wysyła rozkaz `RENDER_DONE`, następnie zbiera potwierdzenia od węzłów, zapisuje wynikowy obrazu do pliku `output.txt` i wyświetla metryki wydajnościowe.

4.2 Opis struktur i funkcji

W naszej implementacji serwera wykorzystaliśmy dużą ilość struktur oraz funkcji, które były kluczowe aby zapewnić poprawne działanie naszego systemu. Z tego powodu omówimy najważniejsze z nich, aby lepiej przybliżyć sposób funkcjonowania serwera.

4.2.1 Struktury danych

- `Rule` - Definiuje pojedynczą zasadę produkcji (zamiany) znaków w procesie derywacji L-systemu.
- `LsysConfig`: Przechowuje definicję L-systemu, parametry derywacji oraz stan początkowy żółwia.
- `Node`: Reprezentuje zarejestrowany węzeł (ID, adres IP/Port, przypisany region, flaga zakończenia).
- `Metrics`: Liczniki statystyczne (liczba wiadomości, unikalne punkty, liczba handoverów).
- `Region`: Struktura definiująca matematyczne granice subregionu na płótnie.

4.2.2 Kluczowe funkcje logiczne

- `derive_lsystem()` - Jej zadaniem jest rozwinięcie prostego aksjomatu w skomplikowany ciąg instrukcji derywacji. Funkcja ta w pętli przeszukuje słowo znak po znaku i zastępuje wybrane litery całymi sekwencjami zgodnie z regułami L-systemu.
- `compute_regions()`- Funkcja ta oblicza optymalny podział dostępnego obszaru do rysowania na prostokątne obszary. Dzięki temu każdy węzeł otrzymuje jasną informację o swoich granicach, co pozwala na łątwą współpracę wielu jednostek nad jednym rysunkiem bez ryzyka nakładania się na siebie.
- `turtle_peek_next()` - Zanim serwer wyśle rozkaz ruchu, funkcja ta oblicza współrzędne, w których żółw znajdzie się po jego wykonaniu. Jeśli nowa pozycja wypada poza granicami obecnego węzła, serwer otrzymuje sygnał, że musi zainicjować procedurę *handoveru*, czyli przekazania zadania sąsiedniemu węzłowi.
- `parse_field()` - Jej celem jest odnalezienie w otrzymanym tekście konkretnych wartości (np. numer kroku czy współrzędne X i Y), ignorując zbędne znaki. To dzięki niej serwer rozumie dane przesyłane przez węzły niezależnie od ich platformy sprzętowej.
- `send_step()` - Formatuje i wysyła pojedyncze zadanie do aktywnego węzła. Poza samą wysyłką, funkcja ta zapisuje dokładny czas wysłania wiadomości, co pozwala mechanizmowi retransmisji wykryć, kiedy węzeł nie odzywa się zbyt długo i należy powtórzyć polecenie.
- `main()` - Centralny ośrodek decyzyjny. Wykorzystuje systemowy mechanizm `select()` do nasłuchiwania komunikatów od wszystkich węzłów na raz oraz pilnowania upływającego czasu. To tutaj zapadają decyzje o tym, czy dany węzeł jeszcze pracuje, czy może należy ponowić wysyłkę z powodu błędu sieci.
- `begin_finish()` -Funkcja zarządzająca procesem bezpiecznego zamykania systemu. Kiedy proces rysowania dobiegnie końca, funkcja ta wysyła sygnał zakończenia do wszystkich węzłów, zbiera od nich ostatnie raporty i dba o to, by gotowy obraz został poprawnie złożony i zapisany w pliku wynikowym.

4.3 Spełnienie wymagań projektowych

Warto też wspomnieć, że nasza implementacja serwera realizuje wszystkie wymagania dotyczące projektu:

- **Wykorzystanie Linux API i C** - Serwer został napisany w czystym języku C z wykorzystaniem standardowych gniazd POSIX (`sys/socket.h`) oraz funkcji `select()` do obsługi asynchronicznej komunikacji nie używając zewnętrznych bibliotek middleware.
- **Niezawodność w związku z UDP** - Zaimplementowaliśmy pełny mechanizm *Stop-and-Wait*. Serwer utrzymuje licznik retransmisji (`retx_count`) oraz zegar (`last_step_send_ms`), co pozwala zagwarantowaĆ dostarczenie każdego symbolu derywacji mimo zawodności warstwy transportowej opartej na protokole UDP.
- **Przypisywanie podregionów** - Serwer pełni rolę jednostki sterującej, która dzieli przestrzeń roboczą. Funkcja `compute_regions()` oraz logika `HANDOVER_REQUEST` zapewniają płynne renderowanie obrazu przez wiele węzłów jednocześnie.
- **Metryki i Diagnostyka** - Struktura `Metrics` pozwala nam na zebranie i wyświetlenie po zakończeniu pracy danych takich jak średni rozmiar pakietu, liczba wiadomości oraz procentowe pokrycie obszaru roboczego.

5 Implementacja Węzła IoT

Węzeł został prez nas zaimplementowany w języku C++ w środowisku emulatora Arduino. W naszym systemie działa on jako jednostka wykonawcza, która interpretuje polecenia derywacji wewnątrz wydzielonego przez Serwer podregionu obszaru roboczego, komunikując się poprzez protokół ALP.

5.1 Procedura działania węzła

Proces działania węzła IoT przebiega w kilku następujących etapach:

1. **Faza Startowa:**
   - Inicjalizacja stosu sieciowego przy użyciu biblioteki `ZsutEthernet`.
   - Wysłanie komunikatu `REGISTER_NODE` do serwera w celu zgłoszenia gotowości, bez czego serwer nie zacznie kolejnych kroków.
2. **Konfiguracja (IDLE)** - Węzeł oczekuje na parametry od serwera. Po odebraniu `ASSIGN_REGION` zapisuje granice swojego obszaru które zostały mu przekazane, a po `INIT_RENDERING` ustawia parametry żółwia (krok, kąt startowy) i czyści lokalny stos.
3. **Oczekiwanie na aktywację** - Nawet po konfiguracji węzeł nie rysuje samoczynnie. Czeka na komunikat `HANDOVER_START`, który informuje go, że żółw właśnie wszedł w jego region i to ten węzeł staje się aktywnym wykonawcą.
4. **Faza Pracy (WORK)** - Po tym jak węzeł odbierze znaki derywacji w komunikatach `RENDER_STEP`:
   - Przelicza on przyszłą pozycję żółwia (`peekNextInt`).
   - Jeśli ruch wykracza poza granice regionu, wysyła `HANDOVER_REQUEST` i wraca do stanu `IDLE`.
   - Jeśli ruch jest wewnątrz granic podregionu, węzeł aktualizuje stan żółwia, wysyła `PLOT` (dla symbolu 'F') oraz potwierdzenie `STEP_DONE`.
5. **Faza Zakończenia** - Po odebraniu komunikatu `RENDER_DONE` od serwera, węzeł przesyła końcowe potwierdzenie i przechodzi w stan spoczynku.

5.2 Opis struktur i funkcji

W celu lepszego wyjaśnienia działania naszych węzłów, wypisaliśmy najważniejsze obiekty oraz funkcje, odpowiadające za prawidłową komunikację z serwerem oraz sam proces rysowania.

5.2.1 Kontekst operacyjny i struktury danych

Nasz węzeł operuje na zestawie zmiennych, które wspólnie tworzą prawidłowo działające urządzenię oraz definiują jego rolę w procesie renderowania:

- `NodeState` - Dwustanowa maszyna stanów (`IDLE` / `WORK`), która zapewnia, że węzeł reaguje na komendy rysowania (`RENDER_STEP`) wyłącznie wtedy, gdy został aktywowany przez serwer komunikatem `HANDOVER_START`.
- `turtleX, turtleY, turtleAngle` - Trójka zmiennych definiująca stan żółwia. Przechowują one aktualne współrzędne oraz kąt nachylenia. Kluczowa jest tu synchronizacja z serwerem przy każdym przekazaniu żółwia do innego węzła (*handover*).
- `regionParts[]` - Tablica, która przechowuje granice podregionu. Dzięki niej urządzenie samo wie, czy może wykonać ruch, czy musi poprosić o pomoc inny węzeł.
- `Stos (stackX, stackY, stackAngle)` - Pozwala żółwiowi na swobodne rysowanie rozgałęzienia fraktala i łatwy powrót do punktu wyjścia (po napotkaniu symbolu zamknięcia nawiasu).
- `handoverSent` - Krytyczna flaga kontrolna, która zapobiega wysłaniu potwierdzenia `STEP_DONE` w momencie, gdy węzeł właśnie oddaje sterowanie. Dzięki temu możemy zapobiec błędom synchronizacji i podwójnemu wykonaniu tego samego kroku przez dwa węzły.

5.2.2 Szczegółowy opis kluczowych funkcji logicznych

- `handleSymbol()` - W tej funkcji zapada decyzja, co oznacza dany znak: czy jest to ruch połączony z rysowaniem (`F`), obrót o zadany kąt (`+`, `-`), czy operacja na stosie. Wywołuje też ona prośbę o *handover*, jeśli ruch prowadzi poza podregion danego węzła.
- `peekNextInt()` - Funkcja ta oblicza docelowe współrzędne ruchu, bez jakielkolwiek zmiany aktualnej pozycji żółwia. Wynik ten jest poddawany natychmiastowej weryfikacji mieszczenia się w granicach, co pozwala na bezpieczne zatrzymanie rysowania na krawędzi przydzielonego obszaru.
- `regionContains()` - Funkcja sprawdzająca, która rozstrzyga, czy dany punkt na płótnie należy do tego konkretnego węzła. Jest wywoływana przed każdym ruchem, aby zapobiec rysowaniu u sąsiada.
- `sendHandoverRequestWithNext()` - Funkcja, która pakujekompletny stan żółwia oraz informację, gdzie dokładnie żółw chce pójść (`next`), i wysyła te dane do serwera. To ona pozwala na działanie systemu rozproszonego, bez wewnętrznych konfliktów.
- `sendPlot()` - Za każdym razem, gdy żółw wykona ruch `F`, ta funkcja przekazuje serwerowi współrzędne punktu, który ma zostać zapełniony na finalnym obrazie.
- `sendStepDone()` - Funkcja ta daje sygnał dla serwera o poprawnym wykonaniu polecenia. Bez niej mechanizm retransmisji serwera uznałby węzeł za niedostępny i zacząłby powtarzać te same rozkazy.
- `parseFieldToBuf()` - Pomocnik parsujący, pozwalajacy węzłowi prawidłowo odczytywać potrzebne dane z protokołu ALP. Dzięki niej węzeł jest odporny na zmiany w kolejności przesyłanych pól wewnątrz wiadomości.

5.3 Spełnienie wymagań projektowych

Implementacja węzła realizuje wytyczne dla systemów wbudowanych IoT:

- **Oszczędność zasobów** - Ze względu na ograniczoną pamięć RAM mikrokontrolera, węzeł nie przechowuje całego obrazu ani derywacji. Zamiast tego przetwarza on znaki jeden po drugim i wysyła punkty `PLOT` na bieżąco. Tam gdzie się tylko dało używaliśmy też całkowitoliczbowych wartości (`int`) aby przyspieszyć obliczenia.

- **Obsługa Handoveru** - Nasz węzeł autonomicznie wykrywa moment wyjścia żółwia poza swój podregion (`HANDOVER_REQUEST`), co jest niezbędne dla działania systemu jako rozproszonego.
- **Niezawodność** - Dzięki wysyłaniu potwierdzeń `STEP_DONE` dla każdego odebranego symbolu, serwer jest w stanie monitorować postęp i wykonywać ewentualne retransmisje w przypadku błędów związanych z UDP.

6 Działanie systemu

Do prawidłowego uruchomienia naszego systemu potrzebowaliśmy 5 maszyn wirtualnych psir23Z, które uruchomiliśmy za pomocą aplikacji Oracle VirtualBox. Wymagane przygotowanie maszyn do pracy wiązało się z wgraniem na nie odpowiednich plików za pomocą połączenia SSH, takich jak:

- Dla **serwera** (1 maszyna):
  - serwer.c - zawierający kod serwera.
  - infile.txt - zawierający położenie początkowe żółwia.
  - ewentualny L_system.txt - zawierający parametry wywołania L-systemu, który jest opcjonalny ze względu na możliwość wpisania tych parametrów z konsoli.
- Dla **węzłów** (4 maszyny) - psir.ino.hex, zawierający kod węzła skompresowany binarnie przez Arduino IDE. Kod każdego z węzłów różni się jedynie adresem IP, adresem MAC oraz identyfikatorem węzła.

6.1 Sposoby wywołania

Zgodnie z wymaganiami naszego projektu, system jesteśmy w stanie uruchomić system w sposób dwojaki:

- Używając pliku .txt zawierającego definicję L-systemu, o następującej strukturze:
  - **Pierwsza linia** - Aksjomat, czyli słowo początkowe derywacji.
  - **Druga linia** - Liczba całkowita określająca całkowitą ilość zasad zawartych w pliku.
  - **Kolejne linie** - Definicje zasad (zgodne z zadeklarowaną wcześniej liczbą). Nasz serwer akceptuje dwa formaty zapisu:
    - `ZNAK,ZAMIENNIK`
    - `ZNAK->ZAMIENNIK`
  - **Ostatnia linia** - Liczba iteracji, określająca ile razy reguły mają zostać zaaplikowane do aksjomatu przed rozpoczęciem renderowania.
  - **Dodatkowo** - Nasz system ignoruje linie puste oraz zaczynające się od znaków `#` lub `//`. Przed przetwarzaniem każdej linii, jest ona oczyszczana również ze spacji i tabulatorów.
- Używając konsoli do wprowadzenia danych o naszym systemie, zostaniemy po kolei zapytani o:
  - **Aksjomat**
  - **Liczbę zasad**
  - **Zasady** w podanej wcześniej liczbie.
  - **Liczbę iteracji**
  - **Dodatkowo** - Dane wprowadzane z konsoli są oczyszczane z białych znaków. W przypadku wykrycia pustego aksjomatu lub błędnego formatu zasady, nasz system informuje o błędzie lub wymusza ponowne podanie poprawnej wartości.

Przy wywołaniu za pomocą, któregokolwiek z tych sposobów konieczny jest również plik `infile.txt`, określający pozycję startową żółwia. Ma on tylko jedną linijkę w postaci - `start=`**wartość_X**:**wartość_Y**:**kąt_nachylenia**.

6.1.1 Wywołanie z plikiem Lsystem.txt

Przy tej metodzie wywołania widzimy jak serwer wczytuje dane o L-systemie z pliku tekstowego, po czym zaczyna czekać na wszystkie 4 węzły. Po tym jak wszystkie się pojawią, rozpoczyna się rendering od węzła pierwszego (jako, że pozycja początkowa się znajdowała się w jego podregionie).

```
student@iot:~/Projekt$ ./obraz_aplikacji L_system.txt
[SERVER] VERSION: v11-retransmit
[SERVER] Loaded L-system config from file: L_system.txt
[SERVER] axiom='FX'
[SERVER] rules=3 iterations=10 (angle=90 fixed)
[SERVER] Start loaded from infile.txt: 35:15:0
[SERVER] L-system length=4094
[SERVER] WAIT_FOR_NODES (need 4)
[SERVER] REGISTER_NODE node-arduino1 (1/4) -> 192.168.56.103:5001
[SERVER] REGISTER_NODE node-arduino2 (2/4) -> 192.168.56.104:5001
[SERVER] REGISTER_NODE node-arduino3 (3/4) -> 192.168.56.105:5001
[SERVER] REGISTER_NODE node-arduino4 (4/4) -> 192.168.56.106:5001
[SERVER] START_RENDERING with active=node-arduino1
[SERVER] HANDOVER -> node-arduino3 (state=32,19 a=90 next=32,20) idx=116
[SERVER] HANDOVER -> node-arduino1 (state=31,20 a=270 next=31,19) idx=124
[SERVER] HANDOVER -> node-arduino3 (state=30,19 a=90 next=30,20) idx=132
```

Rysunek 1: Wywołanie z użyciem pliku L_system.txt

6.1.2 Wywołanie z użyciem konsoli

Korzystając z tego sposobu proces przebiega dokładnie tak samo, z wyjątkiem konieczności wpisywania linijka po linijce wybranych przez nas parametrów.

```
student@iot:~/Projekt$ ./obraz_aplikacji
[SERVER] VERSION: v11-retransmit
[SERVER] Podaj aksjomat: FX
[SERVER] Podaj liczbe zasad: 3
[SERVER] Podaj zasade 1/3 (format: X,REPLACEMENT albo X->REPLACEMENT): X,X+YF+
[SERVER] Podaj zasade 2/3 (format: X,REPLACEMENT albo X->REPLACEMENT): Y,-FX-Y
[SERVER] Podaj zasade 3/3 (format: X,REPLACEMENT albo X->REPLACEMENT): F,F
[SERVER] Podaj liczbe iteracji: 10
[SERVER] Loaded L-system config from stdin
[SERVER] axiom='FX'
[SERVER] rules=3 iterations=10 (angle=90 fixed)
[SERVER] Start loaded from infile.txt: 35:15:0
[SERVER] L-system length=4094
[SERVER] WAIT_FOR_NODES (need 4)
[SERVER] REGISTER_NODE node-arduino1 (1/4) -> 192.168.56.103:5001
[SERVER] REGISTER_NODE node-arduino2 (2/4) -> 192.168.56.104:5001
[SERVER] REGISTER_NODE node-arduino3 (3/4) -> 192.168.56.105:5001
[SERVER] REGISTER_NODE node-arduino4 (4/4) -> 192.168.56.106:5001
[SERVER] START_RENDERING with active=node-arduino1
```

Rysunek 2: Wywołanie z użyciem konsoli

6.2 Wyniki

Co wydaje się być jasne, oba przypadki przedstawione na zdjęciach powyżej opisują dokładnie ten sam system, w związku z czym wyniki końcowe będą takie same.

```
[SERVER] HANDOVER -> node-arduino3 (state=26,19 a=90 next=26,20) idx=1344
[SERVER] HANDOVER -> node-arduino1 (state=27,20 a=270 next=27,19) idx=1352
[SERVER] HANDOVER -> node-arduino3 (state=28,19 a=90 next=28,20) idx=1424
[SERVER] HANDOVER -> node-arduino1 (state=25,20 a=270 next=25,19) idx=1592
[SERVER] HANDOVER -> node-arduino3 (state=24,19 a=90 next=24,20) idx=1616
[SERVER] HANDOVER -> node-arduino1 (state=21,20 a=270 next=21,19) idx=1656
[SERVER] HANDOVER -> node-arduino3 (state=20,19 a=90 next=20,20) idx=1680
[SERVER] FINISH: turtle moved out of canvas after STEP_DONE (pos=41,-1 idx=1773) -> sent RENDER_DONE to all nodes
[SERVER] RENDER_DONE from node-arduino1
[SERVER] RENDER_DONE from node-arduino2
[SERVER] RENDER_DONE from node-arduino3
[SERVER] RENDER_DONE from node-arduino4
[SERVER] Saved output.txt
[SERVER] All nodes done
[SERVER] Messages: 4105
[SERVER] Avg packet size: 31.57 bytes
[SERVER] Handovers: 30
[SERVER] Plots total: 445
[SERVER] Plots unique: 240 / 3200 (7.50%)
```

Rysunek 3: Metryka działania systemu

Metryki, które widzimy powyżej stanowią podsumowanie wydajności i przebiegu naszego procesu rozproszonego renderowania. Możemy z nich wyczytać wiele rzeczy:

- **Messages (4105)** - Całkowita liczba komunikatów UDP wymienionych wewnątrz systemu. Jej wysoka wartość wynika z krokowego charakteru derywacji, gdzie każdy symbol wymaga osobnego pakietu rozkazu oraz potwierdzenia (*ACK*).
- **Avg packet size (31.57 bytes)** - Średni rozmiar ramki danych. Bardzo niska wartość (ok. 32 bajty) świadczy o wysokiej efektywności naszego protokołu ALP.
- **Handovers (30)** - Liczba wykonanych przekazań sterowania między węzłami.
- **Plots total (445)** - Całkowita liczba odebranych żądań narysowania punktu (`PLOT`).
- **Plots unique (240 / 3200)** - Statystyka pokrycia dostępnego obszaru.
  - **240** - Liczba faktycznie postawionych znaków #.
  - **3200** - Całkowita liczba dostępnych pikseli (80×40).
  - **7.50%** - Procent pokrycia obszaru.
- **Przyczyna zakończenia (Status)** - Log pokazuje komunikat `turtle moved out of canvas`, co oznacza, że derywacja została przerwana awaryjnie w kroku 1773, ponieważ żółw próbował wyjść poza zdefiniowane granice świata (osiągnął współrzędną -1). Nie oznacza to jednak, że nasz końcowy obraz będzie dotykał krawędzi tylko, że system planując swój następny ruch zauważył, że do tego dojdzie.

Ostateczny wynikowy obraz, jest częścią krzywej smoka, która nie została w pełni narysowana ze względu wyjście poza krawędź całkowitego obszaru, co pokazuje jak nasz system radzi sobie z taką sytuacją. Proces jest zakończony a wynik zapisany, bez jego utraty, co widzimy na obrazie poniżej.

```
####    ####    #####    ##
 ###   #####   ######   ####
  #### ####### ########  ###
  ####   #####   ######  ###
  ##########################
  #########################
  ### ####################
   ## ############ ### ###
      ##########
      ############
      ### #########
       ## #########
          #########
          #########
          ### ###
           ##  ##
```

Rysunek 4: Końcowy obraz zapisany w pliku output.txt

Dla porównania po wywołaniu bardzo podobnego systemu, tylko z mniejszą liczbą iteracji, działanie systemu kończy się standardowo, jako, że żółw nigdy nie wykracza poza granice obszaru całkowitego.

```
     #####    ##
    ######   ####
    ########  ###
      ######  ###
  ################
 ################
 ###############
   ##### ### ###
  ####
 #####
 ########
   ######
   #####
```

Rysunek 5: Podobny L-system z mniejszą liczbą iteracji

6.3 Testy odporności na błędy

Mimo, że podczas dogłębnego omawiania naszego systemu już z grubsza opowiedzieliśmy o wszystkich zabezpieczeniach to również zawsze warto je przetestować, czym zajmiemy się w tej sekcji.

6.3.1 Zabezpieczenie infile.txt

Tak jak już zaznaczyliśmy wcześniej, system zwraca Error, jeśli struktura pliku wejściowego, nie jest prawidłowa i wypisuje oczekiwany format pliku.

```
[SERVER] rules=3 iterations=8 (angle=90 fixed)
[SERVER] ERROR: bad infile format. Use start=x:y or start=x:y:a
```

Rysunek 6: Błąd infile.txt

6.3.2 Zabezpieczenie przed podawaniem nieprawidłowych parametrów L-sytemu

Bardzo podobnie sytuacja wygląda podczas wywoływania z konsoli, gdzie przy podaniu nieprawidłowej wartości również zwracany jest błąd oraz dawany komentarz jak prawidłowo wprowawdzić dane.

```
[SERVER] Podaj zasade 1/3 (format: X,REPLACEMENT albo X->REPLACEMENT): op'''
[SERVER] ERROR: zly format zasady: 'op''''
```

Rysunek 7: Błąd infile.txt

6.3.3 Rozłączenie jednego z węzłów

Aby zaobserwować prawidłowość działania naszego mechanizmu retransmisji, przeprowadziliśmy test polegający na rozłączeniu jednego z węzłów.

```
D0:1  D1:1  D2:1  D3:1  D4:1  D5:1  D6:1  D7:1  D8:1  D9:1  D10:1 D11:1 D12:1 D13:1

UART
[NODE] STEP idx=4931 sym=]
[NODE] STEP idx=4932 sym=|
[NODE] STEP idx=4933 sym=F
[NODE] move 34,28 -> 35,28 ang=0
[NODE] STEP idx=4934 sym=F
[NODE] move 35,28 -> 36,28 ang=0
[NODE] STEP idx=4935 sym=[
[NODE] STEP idx=4936 sym=+
[NODE] STEP idx=4937 sym=F
[NODE] move 36,28 -> 36,27 ang=270
[NODE] STEP idx=4938 sym=]
[NODE] STEP idx=4939 sym=f
[NODE] move 36,28 -> 37,28 ang=0
[NODE] STEP idx=4940 sym=[
[NODE] STEP idx=4941 sym=-
[NODE] STEP idx=4942 sym=F
[NODE] move 37,28 -> 37,29 ang=90
[NODE] STEP idx=4943 sym=]^CEnd of the aplication (2)
```

Rysunek 8: Wymuszone rozłączenie jednego węzła

W rezultacie na logach serwera, widnieje powtarzająca się 30 razy (tak jak wcześniej wspominaliśmy), próba uzyskania odpowiedzi. Jako, że serwer nigdy jej nie otrzymał, działanie naszego systemu zostało zakończone.

```
[SERVER] HANDOVER -> node-arduino3 (state=40,28 a=180 next=39,28) idx=2066
[SERVER] HANDOVER -> node-arduino4 (state=39,26 a=0 next=40,26) idx=2149
[SERVER] HANDOVER -> node-arduino3 (state=40,26 a=180 next=39,26) idx=2155
[SERVER] HANDOVER -> node-arduino4 (state=39,27 a=0 next=40,27) idx=2179
[SERVER] HANDOVER -> node-arduino3 (state=40,27 a=180 next=39,27) idx=2185
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=1)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=2)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=3)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=4)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=5)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=6)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=7)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=8)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=9)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=10)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=11)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=12)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=13)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=14)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=15)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=16)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=17)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=18)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=19)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=20)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=21)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=22)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=23)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=24)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=25)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=26)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=27)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=28)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=29)
[SERVER] RETX RENDER_STEP idx=4944 to node-arduino3 (try=30)
[SERVER] FINISH: STEP_DONE timeout (too many retransmissions) -> sent RENDER_DONE to all nodes
```

Rysunek 9: Logi serwera próbującego nawiązać kontakt z węzłem

6.3.4 Wykorzystanie wszystkich symboli derywacji L-systemu

Aby zweryfikować implementacje wszystkich symboli derywacji, uruchomiliśmy nasz system dla przykładowego L-systemu, który zawierał każdy z nich, co widać na przedstawionym obrazie.

```
student@iot:~/Projekt$ ./obraz_aplikacji
[SERVER] VERSION: v11-retransmit
[SERVER] Podaj aksjomat: F
[SERVER] Podaj liczbe zasad: 2
[SERVER] Podaj zasade 1/2 (format: X,REPLACEMENT albo X->REPLACEMENT): F,FF[+F]f[-F]|F
[SERVER] Podaj zasade 2/2 (format: X,REPLACEMENT albo X->REPLACEMENT): F+FF,[-f]
[SERVER] Podaj liczbe iteracji: 5
[SERVER] Loaded L-system config from stdin
[SERVER] axiom='F'
[SERVER] rules=2 iterations=5 (angle=90 fixed)
[SERVER] Start loaded from infile.txt: 40:20:90
[SERVER] L-system length=8191
[SERVER] WAIT_FOR_NODES (need 4)
[SERVER] REGISTER_NODE node-arduino1 (1/4) -> 192.168.56.103:5001
[SERVER] REGISTER_NODE node-arduino2 (2/4) -> 192.168.56.104:5001
[SERVER] REGISTER_NODE node-arduino3 (3/4) -> 192.168.56.105:5001
[SERVER] REGISTER_NODE node-arduino4 (4/4) -> 192.168.56.106:5001
[SERVER] START_RENDERING with active=node-arduino4
[SERVER] HANDOVER -> node-arduino3 (state=40,23 a=180 next=39,23) idx=9
[SERVER] HANDOVER -> node-arduino4 (state=39,23 a=0 next=40,23) idx=12
[SERVER] RETX RENDER_STEP idx=45 to node-arduino4 (try=1)
[SERVER] HANDOVER -> node-arduino2 (state=41,20 a=270 next=41,19) idx=51
[SERVER] HANDOVER -> node-arduino4 (state=41,19 a=90 next=41,20) idx=57
```

Rysunek 10: System zdefiniowany używając wszystkich symboli

Jako, że był to losowo wybrany przypadek, z dosyć wysoką ilością iteracji rezultatem był tylko jeden zamalowany piksel.

6.4 Wyjście żółwia poza obszar całkowity

Mimo, że już pokazaliśmy jeden rezultat, który skończył się błędem wykroczenia poza obszar, to warto jeszcze dokładnie wspomnieć o tym kluczowym przypadku, gdyż ze względu na nieprzewidywalną strukture obrazów końcowych, zdarza się on naprawdę często. Dobrym przykładem jest rezultat otrzymany z następującego L-systemu:

1. Aksjomat - **F+F+F+F**
2. Zasada (tylko jedna) - **F,F+F-F-FF+F+F-F**
3. Iteracje - **3**

Rysunek 11: Przykładowy obraz końcowy wyraźnie dotykający krawędzi

7 Podsumowanie

Podsumowując nasz rozproszony system renderujący obrazy na podstawie L-systemów działa prawidłowo. Reaguje on według wytycznych na różne błędy i nietuzinkowe sytuacje, zachowując synchroniczność i bezstratność nawet podczas używania protokołu UDP.